ClutchClip Publisher - upload flow evidence (Project 964206466190)

This PDF contains:

1. Upload interface - the owner adds a video they own or are licensed to use (dashboard > Sources).
2. Review & approval - each clip is checked and approved by the owner before publishing.
3. Settings - the connected YouTube account (OAuth) with a Disconnect button that deletes stored tokens.
4. The complete, working upload script (OAuth 2.0 sign-in + videos.insert resumable upload).

How the upload works:

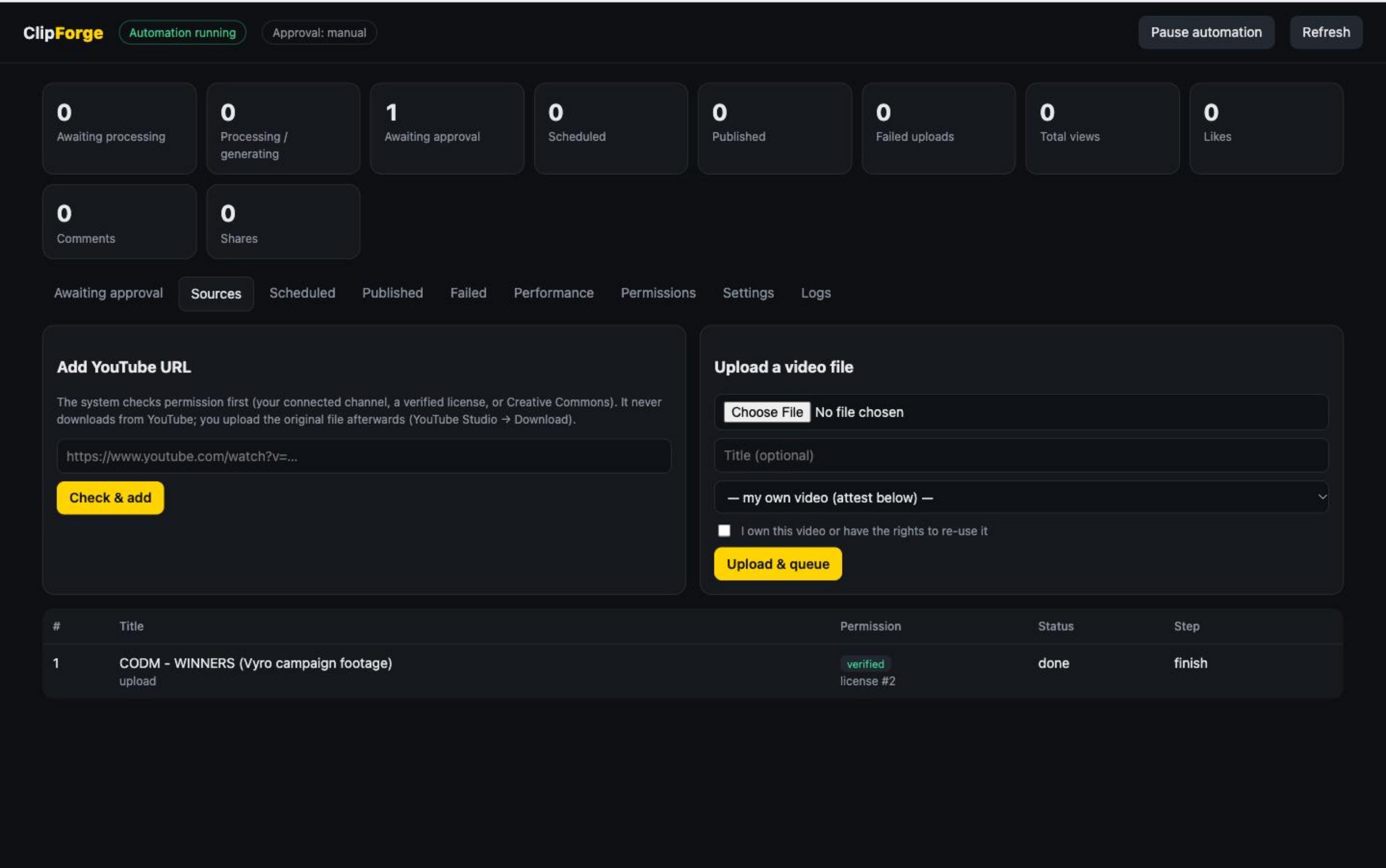
- Owner signs in once with Google OAuth (youtube.upload, youtube.readonly, yt-analytics.readonly).
- Approved clips are queued (max 5/day) and uploaded to the owner's own channel via videos.insert.
- Until this audit is approved, API uploads are private; after approval they are published as Shorts.
- The tool never downloads YouTube videos and never accesses any other user's account.

Website / Privacy / Terms: https://clutchclipofficial-ui.github.io/ (privacy.html, terms.html)

Contact: clutchclip.official@gmail.com Channel: UCMupoVC9nl1Oku3FaAvAH4Q

URL: http://localhost:8000/#sources (private dashboard)

1. Upload interface - add own/licensed video (URL check or file upload with ownership confirmation).



URL: http://localhost:8000/#review (private dashboard)

2. Review & approval - rendered clip, score, editable title/caption/description, Approve / Reject.

ClipForge Automation running Approval: manual Pause automation Refresh



84 story rendered #1 · 25.5s · blur_pad

CODM - WINNERS (Vyro campaign footage) · 0.5–27.9s

Longest continuous segment, captures the entire chaotic sequence from turret setup to the surprise winner, delivering a full arc with high hook and share potential.

Hook (burned into video – editing re-renders)

Metalcore gets sniped from the sky

YouTube title

Metalcore vs Giant Turret in Call of Duty: Mobile – Verdansk S

TikTok caption

Watch Metalcore go solo with a giant turret in Call of Duty: Mobile on Verdansk. Who else loves crazy snipes?

Description

In this intense Call of Duty: Mobile match on Verdansk,

URL: http://localhost:8000/#settings (private dashboard)

3. Settings - YouTube connected via OAuth, Disconnect button (deletes tokens + stats), publishing limits.

ClipForge Automation running Approval: manual Pause automation Refresh

✓ youtube: Clutch Clip Disconnect

Connect YouTube Connect TikTok

approval_mode
"manual"
auto_approve_min_score
70
min_select_score
55
clips_per_source
5
daily_publish_limit
5
publish_times
["10:00","13:00","16:00","19:00","21:30"]
timezone
"Europe/Madrid"
platforms
[]
content_profile
"gaming"
min_clip_s
20
max_clip_s
60
silence_trim
true
tiktok_mode
"inbox"
tiktok_privacy
"SELF_ONLY"
youtube_privacy

4. Complete upload script: youtube_upload.py (lines 1-46 of 126)

```
#!/usr/bin/env python3
"""ClutchClip Publisher - complete YouTube upload script (same flow the app uses).

Uploads ONE approved clip to the channel owner's own YouTube channel through the official
YouTube Data API v3 (videos.insert, resumable upload), after the owner has signed in with
Google OAuth 2.0 (scopes: youtube.upload, youtube.readonly, yt-analytics.readonly).

Usage:
  export GOOGLE_CLIENT_ID=...  GOOGLE_CLIENT_SECRET=...
  python youtube_upload.py --auth                    # one-time sign-in; prints nothing secret
  python youtube_upload.py clip.mp4 --title "..." --description "..." --tags gaming,shorts

Credentials are read from environment variables; the refresh token is stored locally in
~/.clutchclip_youtube_token.json with file mode 600. Nothing is sent anywhere except Google.
"""
import argparse
import http.server
import json
import os
import secrets
import sys
import threading
import urllib.parse
import webbrowser
from pathlib import Path

import httpx

SCOPES = ("https://www.googleapis.com/auth/youtube.upload "
          "https://www.googleapis.com/auth/youtube.readonly "
          "https://www.googleapis.com/auth/yt-analytics.readonly")
REDIRECT = "http://localhost:8765/callback"
TOKEN_FILE = Path.home() / ".clutchclip_youtube_token.json"
CLIENT_ID = os.environ.get("GOOGLE_CLIENT_ID", "")
CLIENT_SECRET = os.environ.get("GOOGLE_CLIENT_SECRET", "")


def authorize() -> None:
    """OAuth 2.0 authorization-code flow with the owner's consent; stores the refresh token."""
    state = secrets.token_urlsafe(16)
    result = {}

    class Handler(http.server.BaseHTTPRequestHandler):
        def do_GET(self):
            q = urllib.parse.parse_qs(urllib.parse.urlparse(self.path).query)
            result.update({k: v[0] for k, v in q.items()})
```

4. Complete upload script: youtube_upload.py (lines 47-92 of 126)

```
47              self.send_response(200)
48              self.end_headers()
49              self.wfile.write(b"Signed in. You can close this tab.")
50
51          def log_message(self, *a):
52              pass
53
54      srv = http.server.HTTPServer(("localhost", 8765), Handler)
55      threading.Thread(target=srv.handle_request, daemon=True).start()
56      url = "https://accounts.google.com/o/oauth2/v2/auth?" + urllib.parse.urlencode({
57          "client_id": CLIENT_ID, "redirect_uri": REDIRECT, "response_type": "code", "scope": SCOPES,
58          "access_type": "offline", "prompt": "consent", "state": state})
59      print("Opening Google sign-in in your browser...")
60      webbrowser.open(url)
61      while "code" not in result and "error" not in result:
62          threading.Event().wait(0.5)
63      if result.get("state") != state or "code" not in result:
64          sys.exit(f"Authorization failed: {result.get('error', 'state mismatch')}")
65      r = httpx.post("https://oauth2.googleapis.com/token", data={
66          "code": result["code"], "client_id": CLIENT_ID, "client_secret": CLIENT_SECRET,
67          "redirect_uri": REDIRECT, "grant_type": "authorization_code"}, timeout=30)
68      r.raise_for_status()
69      TOKEN_FILE.write_text(json.dumps({"refresh_token": r.json()["refresh_token"]}))
70      os.chmod(TOKEN_FILE, 0o600)
71      print("Authorized. Refresh token saved to", TOKEN_FILE)
72
73
74  def access_token() -> str:
75      if not TOKEN_FILE.exists():
76          sys.exit("Not authorized yet - run with --auth first")
77      refresh = json.loads(TOKEN_FILE.read_text())["refresh_token"]
78      r = httpx.post("https://oauth2.googleapis.com/token", data={
79          "client_id": CLIENT_ID, "client_secret": CLIENT_SECRET, "refresh_token": refresh,
80          "grant_type": "refresh_token"}, timeout=30)
81      r.raise_for_status()
82      return r.json()["access_token"]
83
84
85  def upload(path: Path, title: str, description: str, tags: list, privacy: str) -> str:
86      """videos.insert with a resumable upload session (YouTube Data API v3)."""
87      token = access_token()
88      size = path.stat().st_size
89      meta = {"snippet": {"title": title[:100], "description": description[:4900], "tags": tags[:15],
90                          "categoryId": "20"},  # 20 = Gaming
91              "status": {"privacyStatus": privacy, "selfDeclaredMadeForKids": False}}
92      init = httpx.post("https://www.googleapis.com/upload/youtube/v3/videos",
```

4. Complete upload script: youtube_upload.py (lines 93-126 of 126)

```
                        params={"uploadType": "resumable", "part": "snippet,status"}, json=meta,
                        headers={"Authorization": f"Bearer {token}", "X-Upload-Content-Length": str(size),
                                 "X-Upload-Content-Type": "video/mp4"}, timeout=60)
    init.raise_for_status()
    with open(path, "rb") as fh:
        up = httpx.put(init.headers["Location"], content=fh.read(),
                       headers={"Content-Type": "video/mp4"}, timeout=1800)
    up.raise_for_status()
    vid = up.json()["id"]
    return f"https://www.youtube.com/shorts/{vid}"


def main():
    ap = argparse.ArgumentParser(description="Upload one approved clip to your own YouTube channel")
    ap.add_argument("file", nargs="?")
    ap.add_argument("--auth", action="store_true", help="one-time OAuth sign-in")
    ap.add_argument("--title", default="")
    ap.add_argument("--description", default="")
    ap.add_argument("--tags", default="")
    ap.add_argument("--privacy", default="public", choices=["public", "unlisted", "private"])
    a = ap.parse_args()
    if not CLIENT_ID or not CLIENT_SECRET:
        sys.exit("Set GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET")
    if a.auth:
        return authorize()
    if not a.file:
        ap.error("file is required")
    url = upload(Path(a.file), a.title or Path(a.file).stem, a.description,
                 [t.strip() for t in a.tags.split(",") if t.strip()], a.privacy)
    print("Uploaded:", url)


if __name__ == "__main__":
    main()
```